MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

BACHELOR OF INFORMATION SYSTEMS AND TECHNOLOGY (BIST)

EVENING

SYSTEM INTEGRATION AND DEPLOYMENT (IST 3206)

LECTURER: MR. MUCHAKE BRAIN

HOSTEL BOOKING SYSTEM

NAME	STUDENT NUMBER	REGISTRATION NUMBER
MUGISHA RAYAN	2300724192	23/U/24192/EVE
AKULE ROBERT	2300705785	23/U/O5785/EVE
MUSIIME MARTHA TRISHA	2300724223	23/U/24223/PS
AINEBYOONA EVANS	2300725474	23/U/25474/EVE
SSESSANGA NEHEMIAH	2300717751	23/U/17751/PS

2.Description

2.1 Problem statement

Makerere University students currently rely on a fully manual, cash-based hostel booking process. Students are required to physically visit hostel offices, wait in long queues, and make cash payments with no digital confirmation. Custodians manage room inventory using paper registers, which are prone to data loss, duplication errors, and delays. This inefficient approach results in overbooking, lost payment records, and frustration for both students and hostel management staff.

2.2 Proposed Solution

The Hostel Booking System is a full-stack web application built on the MERN stack (MongoDB, Express.js, React, Node.js) that digitises and automates the entire hostel booking lifecycle. The system replaces the manual process with a secure, accessible online platform where students can discover hostels, book rooms, make payments, and receive instant digital receipts all from any device. Custodians gain a centralised dashboard to manage bookings, room inventory, and payments in real time.

The core components integrated within the system include:

- React 19 + Vite Single Page Application (Frontend)
- Node.js + Express.js RESTful API Server (Backend)
- MongoDB Atlas NoSQL Database (Data Layer)
- JWT-based Authentication Middleware (Security)
- Mobile Money / Bank Transfer Payment Gateway (Payments)
- PDF Receipt Generation Service (Receipts)
- Render.com Cloud Hosting (Deployment)

3. Requirements

3.1 Functional Requirements

The following functional requirements define what the integrated system must do across all user roles:

3.1.1 Student Requirements

- Register and log in securely using email/password credentials.
- Browse and filter hostels by location, price, college, and amenities.
- View detailed hostel information including room types, availability, and ratings.
- Complete a multi-step booking form capturing personal, academic, and next-of-kin details.
- Make payments via Mobile Money, Credit/Debit Card, or Bank Transfer.
- Receive an automatic downloadable PDF booking receipt upon successful payment.
- Track booking status (Pending, Confirmed, Cancelled) from a personal dashboard.
- Submit and track maintenance requests for their assigned room.

3.1.2 Custodian Requirements

- Log in to a dedicated custodian dashboard linked to their managed hostel.
- View and process all pending student booking requests.
- Approve or reject bookings and assign specific room numbers to approved students.
- Manage room inventory using an interactive visual floor plan with real-time status updates.
- Add new rooms to the floor plan specifying block, floor, room type, and price.
- Review payment records and confirm payment status before approval.
- Respond to student maintenance requests and update their resolution status.

3.1.3 System / Integration Requirements

- All API endpoints must be protected using JWT middleware for authenticated access.
- The booking service must check for duplicate active bookings before creating a new one.
- Room status (Available, Partially Booked, Booked) must update automatically after each booking approval.
- PDF receipts must be generated client-side using html2canvas and jsPDF upon booking confirmation.
- The frontend must communicate with the backend exclusively via the defined RESTful API (Axios).

3.2 Non-Functional Requirements

Category	Requirement	Implementation Approach
Security	All user data and sessions must be protected.	JWT tokens, bcryptjs password hashing, CORS whitelisting, HTTPS on Render.
Scalability	The system must handle many concurrent users without performance loss.	MongoDB Atlas auto-scaling, stateless REST API, React lazy loading.
Availability	The platform must remain accessible 24/7.	Deployed on Render.com with auto-restart on failure; MongoDB Atlas managed uptime.
Usability	All user roles must navigate the system intuitively on any device.	Responsive React UI; mobile-first CSS; multi-step forms with guided UX.
Maintainability	Codebase must be clean, modular, and easy to extend.	MVC architecture, DRY principles, reusable React components, ESLint, README docs.
Performance	API responses must be fast and the UI must load quickly.	Vite build tool, useMemo for filtering, Mongoose indexed queries.

Data Integrity	Booking and payment records must never be lost or corrupted.	MongoDB ACID-compliant operations, duplicate booking prevention logic.

3.3 Integration Architecture

The system follows a three-tier client-server architecture with a clear separation of concerns between the presentation layer, application logic layer, and data layer:

Frontend Integration (Client Layer)

- Built with React 19 + Vite, organised into feature modules: auth, home, hostels, booking, dashboard, custodian.
- Communicates with the backend via Axios HTTP client configured in api.service.js with JWT-aware interceptors.
- Context API manages global state for authentication, user profile, and toast notifications.

Backend Integration (API Layer)

- Express.js RESTful API with five core route groups: /api/hostels, /api/bookings, /api/users, /api/payments, /api/maintenance.
- MVC pattern strictly enforced: routes delegate to controllers; controllers contain all business logic.
- Shared protect middleware (JWT verification) guards all private routes which are defined once, reused everywhere.
- express-async-handler centralises error handling, preventing sensitive error details from leaking in responses.

Database Integration (Data Layer)

- MongoDB Atlas NoSQL cluster stores five collections: Users, Hostels, Rooms, Bookings, Payments, Reviews.
- Mongoose ODM defines schemas with validation rules; Mongoose populate() resolves references between collections.
- migrateHostels.js and migrateAllHostels.js scripts seed initial hostel and room data into the database.

3.4 Deployment Configuration

The system is deployed on Render.com with two separate services running in production:

Service	Type	URL	Environment
hostel-booking-client	Static Site (Vite Build)	hostel-booking-client.onrender.com	Global CDN
hostel-booking-server	Web Service (Node.js)	hostel-booking-server.onrender.com	Oregon, USA

Environment variables managed securely on Render:

- MONGO_URI - MongoDB Atlas connection string
- JWT_SECRET - Secret key for signing/verifying JSON Web Tokens
- JWT_EXPIRE - Token expiry period (30 days)
- CLIENT_URL - Whitelisted frontend origin for CORS protection
- PORT - Express server listening port

3.5 Security Integration Requirements

- Password hashing: bcryptjs salts and hashes all passwords before storage; plain-text passwords are never persisted.
- JWT Authentication: every protected API route verifies the Bearer token in the Authorization header before processing.
- CORS Protection: the backend only accepts requests from the whitelisted CLIENT_URL, blocking all unauthorised origins.
- Secure Error Handling: express-async-handler and custom error middleware prevent stack traces and internal details from appearing in API responses.
- Environment Isolation: all secrets are stored in .env files excluded from version control; Render injects them at runtime.

3.6 Testing and Validation Requirements

- Manual API testing conducted via Postman for all endpoints: GET /api/hostels, POST /api/bookings, PUT /:id/cancel, etc.

- Error scenario testing: duplicate booking prevention verified by attempting two concurrent bookings for the same student.
- Authentication testing: protected routes verified to return HTTP 401 when accessed without a valid token.
- Jest + Supertest configured for automated backend unit and integration tests; test suite located in server/__tests__/.
- Frontend integration verified by end-to-end user flows: Register → Search Hostel → Book Room → Pay → Download Receipt.

Conclusion

The System Integration and Deployment section describes how the Hostel Booking System brings together its frontend, backend, database, payment, and deployment components into a single cohesive platform. By following the MERN stack architecture with the MVC design pattern, the system achieves a clean separation of concerns that makes each layer independently maintainable and scalable. Security is enforced at every boundary through JWT authentication, password hashing, CORS protection, and secure environment variable management. The

deployment on Render.com provides a reliable, always-available production environment accessible to all Makerere University students and hostel custodians at any time from any device.